DD2438 Assignment 2: Multi-Agent Navigation via Space-Time Reservation and Reactive Pedestrian Shielding

GROUP 3
Kerun Liu Marceau Messaussier
kerun@kth.se marceaum@kth.se

Abstract

We present a deadlock-free multi-agent navigation architecture that achieves a 100% completion rate across all tested topologies by combining space-time reservation planning with reactive pedestrian shielding. As autonomous fleets increasingly scale in modern logistics and urban mobility, this research is important because it addresses a critical algorithmic bottleneck: relying solely on traditional single-agent pathfinding inevitably leads to catastrophic deadlocks at spatial bottlenecks, while purely reactive collision avoidance fails to optimize global task efficiency. To resolve these challenges, we implemented two space-time planners: a Hierarchical Cooperative A* (HCA*) planner utilizing a 3D space-time reservation table, and a greedy lightweight variant in which a geometrical path is first computed before reserving the timing along it. Both mathematically guarantee collision-free global routes among peers. For physical execution, we deployed vehicle-specific kinematic controllers (the Stanley controller for cars and a proportional derivative (PD) controller for drones), augmented by a reactive Artificial Potential Field (APF) to safely evade unpredictable pedestrians. Furthermore, we introduced a greedy Sequential Path-Chaining algorithm to address the "bakery" load-balancing problem when objectives outnumber available agents. Experimental evaluations across diverse competition topologies demonstrate our system's absolute stability. Ultimately, this approach highlights the practical importance of prioritizing space-time reservations and reactive shielding to guarantee systemic reliability in highly constrained real-world and simulated multi-agent tasks.

1 Introduction

Coordinating autonomous fleets in dynamic environments is fundamentally challenging: single-agent pathfinding produces deadlocks at spatial bottlenecks, dynamic obstacles can hinder paths, purely reactive avoidance cannot guarantee global efficiency, and team-based task allocation adds a further layer of combinatorial complexity. As autonomous fleets scale across modern logistics, urban mobility, and warehouse automation, solving this coordination problem becomes a critical prerequisite for safe and efficient real-world deployment. This paper addresses these three challenges for two agent classes — non-holonomic cars and holonomic drones — navigating through static obstacles and unpredictable dynamic pedestrians. To do so, we propose a hybrid architecture that couples a space-time planner and a controller augmented by a reactive logic for pedestrian evasion.

We address two distinct classes of autonomous agents navigating through highly constrained environments: traffic cars (Problem 1) and traffic drones (Problem 2) with time reservation algorithms and proper controllers (Stanley and proportional derivative). The traffic cars are modeled as non-holonomic agents, requiring continuous forward motion and bounded steering angles, whereas the drones function as holonomic agents capable of instantaneous omnidirectional translation. Both vehicle types must navigate dense, shared map topologies—such as four-way intersections, merging highway on-ramps, and narrow bottlenecks. To overcome these challenges we use time reservation algorithms (HCA* for drones, a greedy lightweight variant for cars) with vehicle-specific controllers (Stanley for cars, PD for drones).

Beyond static infrastructure, roaming pedestrians introduce dynamic uncertainties that pre-computed global paths cannot anticipate — motivating a supplementary reactive layer. Pedestrians trajectories are locally predictable over short horizons but globally unpredictable, meaning agents must rely on reactive mechanisms to evade without catastrophically deviating from their reserved routes. To handle this, we augment both controllers with an Artificial Potential Field (APF) that generates repulsive forces around nearby pedestrians. While this approach works reliably for holonomic drones — which can move in any direction — it remains a limitation for non-holonomic cars, whose bounded steering angles prevent systematic evasion and can result in slowdowns.

The "bakery" map variations elevate the operational challenge from basic collision-free navigation to complex fleet load balancing. In these scenarios, vehicles are grouped into cooperative teams responsible for serving a shared pool of spatially distributed customers (goals). This shifts the paradigm from static point-to-point routing to multi-task allocation, requiring a system

that assigns, based on distances between tasks, available vehicles to unvisited locations to ensure complete objective coverage and minimize the overall fleet makespan.

1.1 Contribution

This subsection outlines our hybrid architecture's core algorithmic contributions, which strictly decouple global coordination from local execution to achieve a 100% deadlock-free completion rate.

Specifically, our system introduces three core mechanisms:

- **Cooperative Spacetime Scheduling:** A HCA* planner (x, y, t) enforcing temporal reservations and terminal state permanence ($t \to \infty$) to guarantee collision-free global routes and eliminate peer-to-peer deadlocks at spatial bottlenecks.
- **Kinematic Tracking & Reactive Evasion:** Integration of vehicle-specific controllers (Stanley for cars, PD for drones) coupled with an Artificial Potential Field (APF). This provides a reactive mathematical shield to bypass unpredictable pedestrians without triggering expensive global re-planning.
- **Sequential Path-Chaining Allocation:** A deterministic routing algorithm for complex "bakery" scenarios. By inheriting spatial coordinates and temporal states (t_n) from preceding targets, agents seamlessly concatenate multiple goals into a single, conflict-free spacetime trajectory.

1.2 Outline

The remainder of this report is structured as follows. Section 2 reviews the foundational literature regarding multi-agent cooperative planning, kinematic feasibility, and trajectory tracking. Section 3 provides a comprehensive explanation of our proposed hybrid architecture, detailing the distinct control logic for non-holonomic cars and holonomic drones, along with the implementation specifics. Section 4 outlines the experimental setup and analyzes our system's performance metrics relative to the broader class cohort, including a critical examination of our algorithmic trade-offs. Finally, Section 5 concludes the paper with a summary of our findings and identifies potential avenues for future optimization.

2 Related Work

This section surveys the five algorithmic pillars that motivate each layer of our hybrid architecture: heuristic search foundations (particularly, A*

algorithm), cooperative space-time planning, path-velocity decomposition, kinematic-aware trajectory generation, and local reactive avoidance. Existing literature treats these as separate problems; our contribution is to combine space-time reservation with heterogeneous agent controllers and a reactive pedestrian shield into a single deployable system.

The A* algorithm provides a formal framework for heuristic pathfinding, guaranteeing optimal solutions when the cost estimates are admissible. As shown by [2], the efficiency of A* critically depends on the choice of heuristic function $h(n)$. In the context of autonomous traffic, A* serves as the backbone for static navigation, generating baseline paths that respect the environment's topology while ignoring dynamic agents. However, A* does not account for multi-agent interactions or temporal constraints, which can lead to collisions or deadlocks in shared environments. For our implementation, we used an inverted Dijkstra map as the heuristic.

Cooperative pathfinding resolves agent-to-agent conflicts by extending the search space into a temporal dimension using a reservation mechanism. As introduced by [6], the Cooperative A* (CA*) and Hierarchical Cooperative A* (HCA*) algorithms rely on a space-time reservation table (x, y, t). By planning agents sequentially, these methods ensure that no two vehicles occupy the same cell simultaneously. HCA* further optimizes this process by using a precomputed 2D distance map as a heuristic to guide the 3D search, reducing the "blind" temporal exploration typical of basic cooperative solvers. However, this approach remains computationally expensive and can lead to rigid behaviors, this rigidity may be a problem with unexpected obstacles like pedestrians. We applied this technique for drones, as it is flexible despite requiring many iterations. For cars, we employ a computationally lightweight alternative that achieves comparable results.

Decoupling path geometry from velocity profiling reduces the computational burden of high-dimensional planning. Following the Path-Velocity Decomposition principle [4], a fixed spatial path is first determined, and timing along that path is optimized separately. For traffic cars, this converts a 3D (x, y, t) problem into a 1D scheduling task, allowing vehicles to wait at intersections or adjust speed without leaving their assigned rail. This approach was adopted for cars to reduce computational cost, accommodate two different hardware setups, and avoid the chaotic situations that HCA* can produce along with pedestrians. Unlike drones, which can exploit free space more flexibly, cars can easily become congested on their rails; unexpected events, such as pedestrian interactions, would then disrupt the system entirely if HCA* were used directly.

Adapting discrete search to real vehicles requires accounting for kinematic constraints. Hybrid A* [1] addresses this by using motion primitives to gen-

erate smooth, feasible trajectories instead of jagged grid-based paths. We adopt a computationally efficient approximation of this approach to balance feasibility and performance. However, generating a feasible path alone is insufficient, as accurate execution under vehicle dynamics remains challenging.

The Stanley Controller provides a stable, nonlinear feedback law that minimizes both heading and lateral errors during trajectory tracking. As described by [3], it uses a look-ahead distance, a point further on the trajectory to head to. Its exponential convergence to the path makes it well-suited for our scenarios, where strict adherence to a "rail" is critical to avoid collisions with infrastructure. Most importantly since our pipeline relies on spatial-temporal reservations, maintaining the vehicle precisely on its assigned path is essential. However, path adherence alone is insufficient; agents must also evade pedestrians locally.

Artificial potential fields offer a reactive layer for local obstacle avoidance by exerting repulsive forces that divert the vehicle from immediate threats. While global planners handle coordination, [5] introduced the concept of using attractive forces for goals and repulsive forces for obstacles. Here, if a pedestrian appears, the vehicle can "nudge" its position sideways based on the repulsion intensity, making a compromise between safety and complete recalculation which would be too costly.

3 Proposed method

This section presents our approach to solving the multi-agent pathfinding problem described in the above introduction. Our pipeline is composed of two or three main stages: Greedy Task Allocation 3.1 if needed, Space-time planner 3.2, and then we describe the controllers used respectively for the cars and the drones in section Controllers 3.3.

3.1 Global Task Allocation for bakery maps

In bakery maps, a greedy Dijkstra-based strategy with spatial and capacity constraints is used to distribute tasks efficiently across the fleet.

Within teams, vehicles self-assign targets sequentially by evaluating Dijkstra distance maps. To ensure localized operation and workload balance, an agent ceases route extension if the nearest target exceeds a threshold $\mathcal{H}_{max} = 100m$ or if a limit of $N = 3$ targets is reached (hyper-parameters empirically defined). This mechanism prevents inefficient transitions between distant clusters, while, if necessary, a final "clean-up" agent ensures that all remaining tasks are completed regardless of these constraints.

3.2 Space-time path planning

In this section, we describe our global space-time planners, which operate over both spatial and temporal dimensions. For drones, we use HCA* directly as introduced in [6]. However, for cars we adopt a more greedy pipeline: we first perform spatial planning with reservations (Section 3.2.1), then apply temporal scheduling along the resulting paths (Section 3.2.2). This results in a lightweight and stable HCA*.

3.2.1 Fast Kinematic-Aware Path Planning

We generate physically feasible routes using a lattice-inspired A* search that retains continuous heading at each node. Motion primitives provide a lightweight alternative to the three-dimensional grid-based search used in Hybrid A* [1] (x, y, θ).

The search is fast and used for both, the drones and the cars. The search operates on a 2D (x, y) grid, where each node stores a continuous orientation to expand a fixed set of steering actions. Each node expands neighbors using steering angles of $-30°, -15°, 0°, 15°$, and $30°$. A drawback of this method is that inconsistencies may occur during path reconstruction. However, the Dijkstra heuristic, by guiding the search, ensures trajectories remain smooth enough for both the Stanley controller (cars) and the HCA* temporal planner (drones), as illustrated in Fig. 1. Given the nature of the drone, a rough A* should be sufficient even if we used this version of A* for our experiments.

3.2.2 Temporal Scheduling via Space-Time Reservation

This section concerns cars only, once a geometric path is computed, inter-vehicle collisions are resolved through a sequential temporal planning step. We then compute an optimal, collision-free timing profile along the rail. In addition, a temporal safety buffer is enforced to maintain sufficient distance between vehicles.

The fixed path is treated as a "rail," along which the vehicle's progression—either moving at v_{max} or waiting—is optimized to safely traverse shared areas. Following the Path-Velocity Decomposition principle [4], we define a search space over (index, t), where the index represents a position along the path, and apply A* to compute an optimal timing profile.

Safety is ensured via a temporal buffer that accounts for execution uncertainties. Since vehicles cannot stop instantly, each planned state reserves a range of indices before and after the vehicle. For example, if a car is scheduled at index 50 at time $t = 10$, indices 40 to 51 are also kept free. This creates a protective "bubble" behind the vehicle (See colorful tubes in Fig. 1 for illustration), preventing collisions even with tracking errors or local disturbances (see Section 3.3). The buffer is biased behind the vehicle because

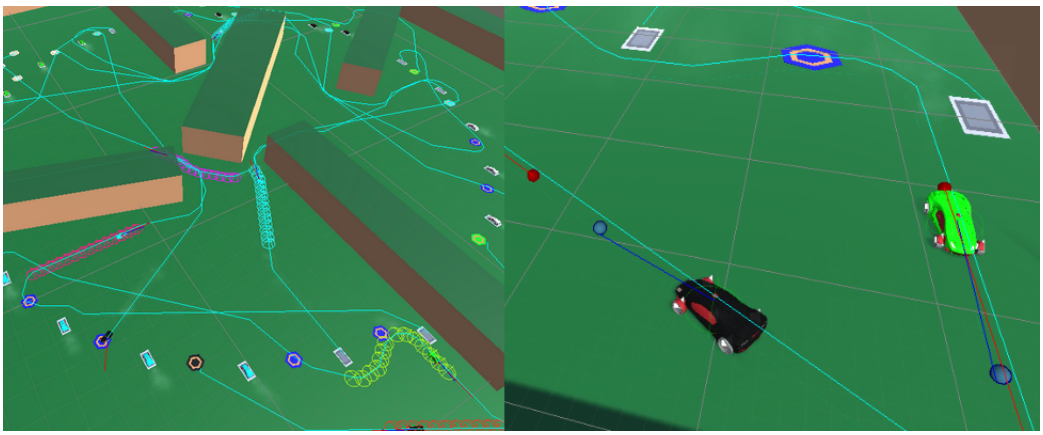

Figure 1: Planned paths for the cars on the *bakery 2* map. Cyan lines represent the geometric "rails," while colored tubes denote the time windows assigned to each car (one color per car). The left figure shows an overview of the scene, while the right provides a zoomed view highlighting the Blue Point (spatial look-ahead ghost) and the Red Point (temporal ghost). Black car is late while green one in ahead in timings.

it is more likely to be slowed by obstacles and pedestrians than to exceed the planned speed.

3.3 Controllers

This section summarizes the execution layer for both cars and drones. Cars are controlled using a Stanley-based approach, while drones rely on a PD controller. In both cases, a reactive Artificial Potential Field (APF) layer handles local pedestrian avoidance. The controllers ensure adherence to planned space-time paths while maintaining robustness to dynamic obstacles and execution uncertainties.

3.3.1 Car controller details

This subsection details the execution layer for non-holonomic cars, which must simultaneously track a geometric path and respect a reserved time slot. To achieve this, we decouple spatial guidance from temporal tracking via a "Red Point / Blue Point" scheme, and augment the controller with a reactive potential field for pedestrian avoidance. A recovery maneuver handles the residual cases where the vehicle becomes fully blocked.

The execution layer combines a Stanley controller for steering with a timing-profile-based speed law, augmented by a potential field for pedestrian avoidance. The two mechanisms are decoupled, as illustrated in Fig. 1 through the blue/red points.

- **Blue Point (spatial ghost):** projected a fixed look-ahead distance d ahead of the vehicle along the geometric rail. The Stanley controller minimizes the heading error and lateral offset to this point.
- **Red Point (temporal ghost):** advances along the rail at the reserved time profile. The throttle is governed by the signed distance Δd to this ghost: if the car is late, Δd grows and throttle increases; if the car is early, Δd shrinks and the car decelerates. This implements a 1D speed controller directly along the rail.

Together, the Blue Point controls *where* the car steers (lateral tracking) while the Red Point controls *when* it moves (temporal tracking), ensuring simultaneous adherence to both the geometric rail and the reserved time slot. The car sticks to the path. The sole exception occurs when a pedestrian is detected, temporarily deflecting the Blue Point laterally.

Pedestrian avoidance uses a lateral repulsive force. While following its rail, nearby agents are monitored. When a pedestrian is detected, the Blue Point is shifted laterally, causing the Stanley Controller to steer around the obstacle without re-planning. Once clear, the Blue Point pulls the car back to its path. In crowded scenarios, excessive forces can push the car off-track or cause it to get stuck.

To handle deadlocks caused by pedestrians, the system monitors progress and triggers a reversal-based recovery maneuver. A persistence timer detects when a vehicle cannot reach its temporal target. If the distance to the Red Point remains large while velocity is near zero for over 0.8 seconds, the agent is flagged as `isStuck`. This activates a recovery state where the vehicle reverses and inverts its steering, effectively resetting its position to attempt a new approach.

3.3.2 Drone controller details

This subsection details the kinematic execution and reactive control architecture for the holonomic traffic drones, explicitly defining the mathematical integration of PD tracking for global routes and APF for local evasion.

To bridge the gap between the discrete spacetime nodes and continuous physical movement, we introduced a temporal interpolation mechanism. The system calculates a virtual target position—referred to as the "Time Ghost"—by linearly interpolating between the discrete A* waypoints based on the elapsed global simulation time. A Proportional-Derivative (PD) controller is then utilized to generate the necessary two-axis thrust vectors. Specifically, the attractive force vector F_{attr} is defined as:

$$F_{attr} = K_p e(t) + K_d \frac{de(t)}{dt}$$

where $e(t) = p_{ghost}(t) - p_{drone}(t)$ represents the positional error, with K_p providing a strong attractive force toward the virtual target and K_d acting as a necessary damper to eliminate pendulum-like oscillations upon approaching the goal. Furthermore, to resolve kinematic desynchronization—a critical edge case where the virtual time progresses faster than the drone's physical maximum velocity—the temporal progression is strictly halted at the final node. The simulation only registers the task as completed when the drone's physical distance to the goal falls within a strict predefined tolerance.

While the macro-level Spacetime A* planner guarantees the avoidance of static infrastructure and other pre-planned agents, it fundamentally cannot account for globally unpredictable pedestrians. To address this dynamic uncertainty, we integrated an Artificial Potential Field (APF) as a reactive secondary control layer. By continuously monitoring the relative distance between the drone and nearby pedestrians, the system generates a repulsive force vector F_{rep} that scales exponentially as the distance d decreases:

$$F_{rep} = \begin{cases} \eta \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{1}{d^2} \nabla d & \text{if } d \leq d_0 \\ 0 & \text{if } d > d_0 \end{cases}$$

where d_0 is the predefined safety influence radius, η is the repulsion gain constant, and ∇d is the unit direction vector pointing away from the pedestrian.

The final kinematic control input is the dynamic superposition of these vectors: $F_{total} = F_{attr} + \sum F_{rep}$. Consequently, the drone is capable of organically sliding around pedestrians to evade immediate collisions. Once the dynamic obstacle is cleared ($d > d_0$), the APF force drops to zero, and the PD controller's restorative force immediately pulls the drone back onto its globally reserved spacetime trajectory, achieving seamless local avoidance without the computational overhead of global re-planning. Figure 2 shows that the drone can avoid the pedestrian via the repulsive force.

3.4 Implementation

The software architecture uses asynchronous multi-threading and optimized state matrices to maintain high simulation frame rates during intensive calculations, especially for HCA*. To prevent Unity's main thread from freezing, all pathfinding and heuristic generation (Reverse Breadth-First Search) are executed on background threads. A key engineering optimization involved implementing a strict `inQueue` state matrix for heuristic generation; by tracking node visitation, we eliminated exponential queue explosions and bounded computation time, preventing out-of-memory exceptions. Once a trajectory is solved, it is locked in a global 3D Reservation System that stores spheres at specific time-space positions, acting as a constraint for subsequent agents.

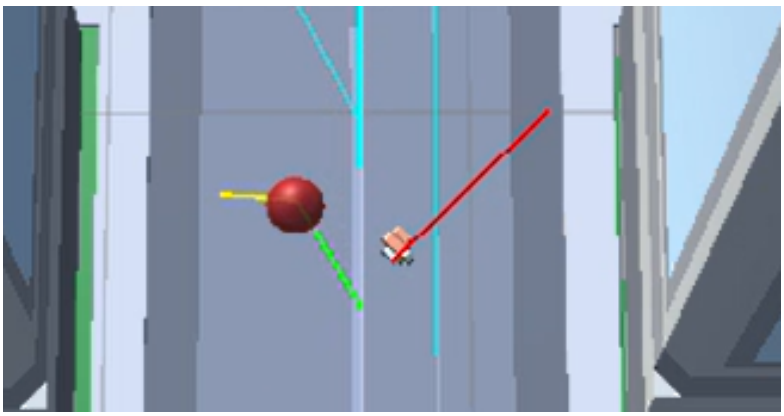

Figure 2: Path tracking with APF for drones to avoid pedestrians.

For bakery scenarios where goals outnumber agents, a Sequential Path-Chaining framework concatenates multiple goals into a single deadlock-free trajectory. When goals outnumber available agents, the system partitions objectives and assigns them to specific vehicles. An agent then performs consecutive spacetime queries: the arrival time t_1 and coordinates of the first goal serve as the initial state for the next search. This process continues until the entire chain of goals is resolved and committed to the global `ReservationTable` as one continuous deployment, ensuring that multi-stop routes remain deadlock-free.

4 Experimental results

This section presents the empirical evaluation of our proposed hybrid architecture. The primary objective is to benchmark our global makespan against the class cohort while analyzing the algorithmic trade-offs between raw execution speed and multi-agent robustness.

4.1 Experimental setup

This subsection outlines the simulation environment, map topologies diversity, and performance metrics utilized to benchmark our autonomous fleet.

The experiments were conducted using the provided Unity simulation framework, testing both non-holonomic cars (P1) and holonomic drones (P2). The test suite ranges from unobstructed domains (`open`) to severe spatial bottlenecks (`highway`) and complex team-based task allocations (`bakeries` and `city_bakery`). The primary performance metric is the global completion time required to clear all goals, allowing us to evaluate our system's reliability under dynamic uncertainty against the broader class dataset.

4.2 Analysis of Outcome

An analysis of the competitive data reveals a clear trade-off between absolute deadlock resilience and kinematic throughput. While our architecture successfully passed all maps, demonstrating high system stability, our raw completion times were suboptimal compared to top-performing groups using predictive kinematics.

Table 1 summarizes the performance of our architecture (Group 3) relative to the class averages and the absolute best times recorded by the top-performing teams.

	P1 Cars (Time in s)			P2 Drones (Time in s)		
Map Scenario	**Our Group (G3)**	**Class Avg**	**Best Time [Group]**	**Our Group (G3)**	**Class Avg**	**Best Time [Group]**
open	101.04	90.38	**30.60** [G14]	158.46	83.14	**17.58** [G16]
open_blocks4	**116.12**	135.54	**56.00** [G14]	127.90	97.57	**30.24** [G11]
intersection4	**227.10**	244.82	126.16 [G10]	512.45	207.29	**67.68** [G11]
highway4	**165.14**	215.07	105.98 [G17]	653.53	212.35	**55.86** [G11]
onramp4	359.06	352.72	**218.18** [G10]	586.42	304.88	**104.32** [G17]
bakeries4	144.94	140.98	**69.62** [G17]	199.86	137.16	**45.20** [G11]
city_bakery4	**249.04**	302.14	169.14 [G15]	573.08	272.54	**96.62** [G16]

Table 1: Comparison of global completion times for all tested maps. Bold values in the 'Our Group' column highlight instances where our system outperformed the class average.

Our space-time planner performs reliably in constrained topologies but is slower in open, high-speed scenarios. In `open_blocks4`, `highway4`, and `city_bakery4`, our completion times were below the class average, suggesting that space-time reservation handles bottlenecks well and avoiding the deadlocks that caused some groups to never complete the map. In contrast, on the unobstructed `open` map and the merging `onramp4` scenario, our cars were slower than average. This is primarily due to a deliberate trade-off: our timing profile is conservative to maintain schedule adherence when pedestrians cause slowdowns, which comes at the cost of raw speed in unconstrained environments. This is a major flaw of our rigid approach.

Furthermore, our P2 Drone makespans were consistently higher than the class average. These extended delays are primarily caused by an overly conservative Artificial Potential Field (APF). Because the APF generates excessively large repulsive vectors, our drones frequently yield or take wide detours around pedestrians, forcing a reduction in baseline velocity to maintain stability. A comparative analysis against top-performing teams, specifically Group 16, provides profound insights into why their method was superior in terms of speed. Post-competition reviews revealed that Group 16 heavily relied on predictive geometric algorithms—namely the Velocity Obstacle (VO) formulation operating in the velocity space. Unlike our APF, which

is fundamentally a reactive approach that resolves conflicts only when distance thresholds are breached (resulting in conservative deceleration), the VO method is purely predictive. By calculating forbidden velocity cones and selecting optimal acceleration vectors outside these boundaries, their drones maintained high velocities and bypassed pedestrians with minimal deviation. This exposes a crucial takeaway: while our macro-level HCA* provides mathematically guaranteed deadlock resilience, our micro-level APF is too primitive for high-speed dynamic evasion. Future iterations must transition to predictive kinematics like VO or Optimal Reciprocal Collision Avoidance (ORCA) to achieve high-throughput performance.

5 Summary and Conclusions

This section summarizes our contributions and their empirical trade-offs, and outlines directions for future work. We presented a three-layer hybrid architecture achieving a 100% completion rate with zero deadlocks across all seven tested topologies. This reliability stems from three core contributions: a space-time planner enforcing space-time reservations to guarantee deadlock-free global routes; vehicle-specific controllers (Stanley for cars, PD for drones) decoupled via the Red Point / Blue Point scheme; and a reactive APF layer for local pedestrian evasion without global re-planning. For multi-goal scenarios, Sequential Path-Chaining concatenates consecutive spacetime queries into a single conflict-free trajectory. This robustness came at a throughput cost: our drone makespans consistently exceeded the class average. This gap stems from the reactive nature of the APF, which resolves conflicts only once distance thresholds are breached, unlike the Velocity Obstacle formulation used by top-performing groups. Future iterations should therefore replace the APF with VO or ORCA to preserve our deadlock-free guarantees while closing this performance gap.

References

[1] Dmitri Dolgov, Sebastian Thrun, Michael Montemerlo, and James Diebel. Practical search-techniques in path-planning for autonomous driving. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2008. Paper presenting Hybrid A*.

[2] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. Initial paper presenting A*.

[3] Gabriel M. Hoffmann, Claire J. Tomlin, Michael Montemerlo, and Sebastian Thrun. Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and race-related considerations. *Journal of Dynamic Systems, Measurement, and Control*, 129(2):186–195, 2007.

[4] Kamal Kant and Steven W. Zucker. Toward efficient planning of safe trajectories in dynamic environments. *The International Journal of Robotics Research*, 5(3):77–89, 1986.

[5] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, 5(1):90–98, 1986.

[6] David Silver. Cooperative pathfinding. In *Proceedings of the First Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE-05)*, pages 117–122. AAAI Press, 2005.